

RelPT: reusable PPO preparation through relational artifact control

RelPT is the system layer in Prime_TTT that makes post-training artifacts reusable. It records prompts, rollouts, rewards, logprobs, advantages, batches, policies, and updates as append-only relational rows, then computes only the missing delta before PPO.

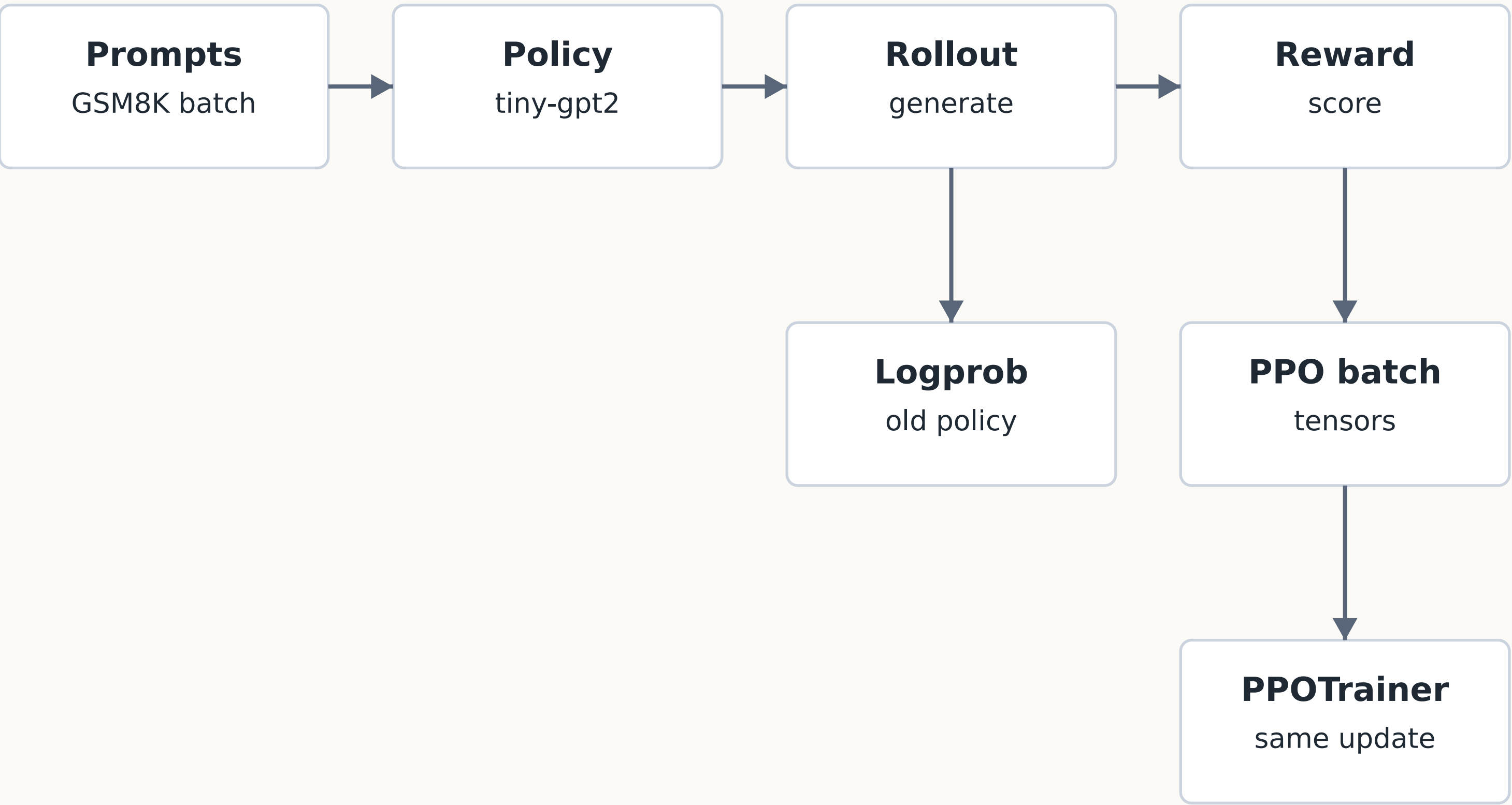
The short version

Baseline preparation redoes generation, reward scoring, and logprob work. RelPT first queries the artifact tables. Existing rows are reused; only absent rows trigger expensive executors. PPOTrainer still receives the same batch shape.

What this report illustrates

- Where RelPT sits around rollout, reward, logprob, and PPOTrainer.
- How SQL joins turn retries and partial recomputation into delta work.
- Which tables carry lineage for reproducibility and recovery.
- What the latest same-framework TRL PPO measurement shows.
- Why the result is a systems efficiency signal, not a model-quality claim.

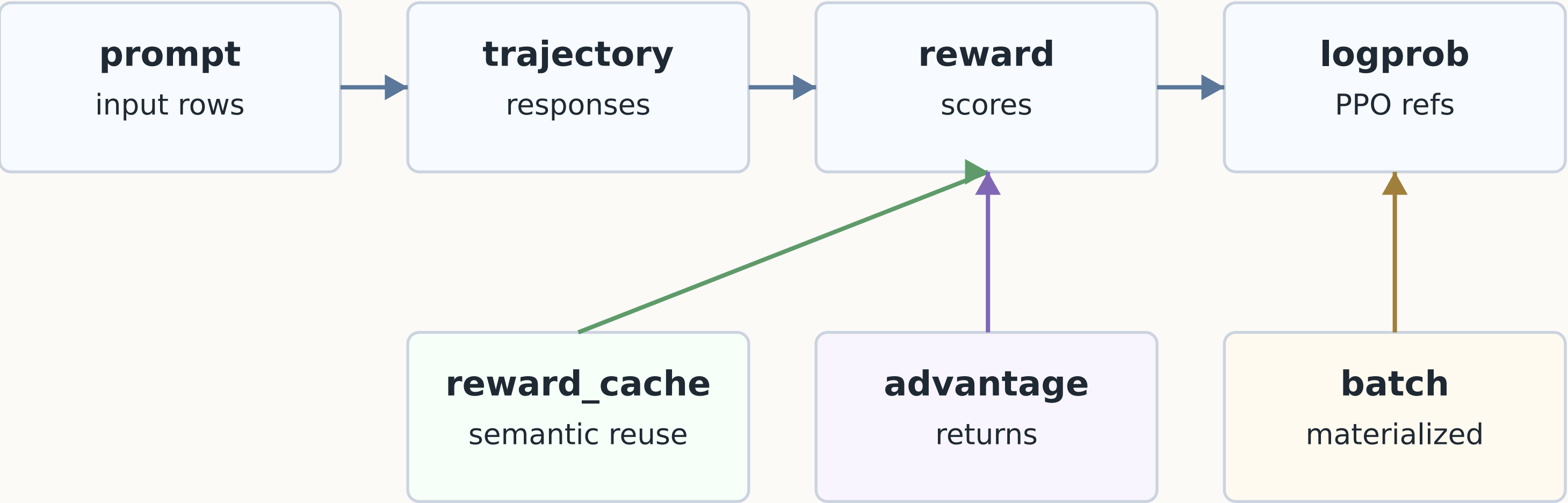
Without RelPT: retries repeat work



Why this wastes work

A failed run, retry, cache miss investigation, or change from K=4 to K=6 can ask for almost the same batch again. A naive pipeline has no durable artifact map, so it often regenerates responses and rescans rewards even when most rows already exist.

With RelPT: query first, execute only the delta



Delta planner worklists

rollout_requests	prompts with too few usable trajectories for the current policy
reward_traj_ids	trajectories whose reward is not found in reward or reward_cache
logprob_traj_ids	trajectories missing old-policy logprob rows
advantage_traj_ids	rewarded trajectories missing returns and advantages

Tables make lineage, reuse, and recovery explicit

prompt	input text, dataset metadata, compact labels
policy	policy versions before and after PPO updates
trajectory	rollout artifacts keyed by prompt and policy
reward	score for a trajectory under a reward model
reward_cache	deterministic reuse by reward model, prompt, response hash
logprob	old-policy logprob rows required by PPO
advantage	return and advantage rows
batch	materialized PPO batch membership
update_record	input policy, output policy, batch, trainer lineage
metric	runtime, row-count, reward, and system metrics

Join semantics

The important join is not only by trajectory id. reward_cache joins on reward_model_id, prompt_id, and response_hash, so deterministic rewards can be reused across policy versions when the same prompt-response pair appears again. Other LEFT JOINs identify missing rollouts, rewards, logprobs, and advantages.

Two implementations, one control-layer idea

Full SQLite RelPT prototype

`relpt.py` creates the append-only schema and implements `RelPT.plan\_ppo\_batch` plus `RelPT.prepare\_ppo\_batch`. Rollout, reward, logprob, and PPO training are black-box executors, which makes the relational contract testable without GPU LLM frameworks.

Same-framework TRL PPO comparison

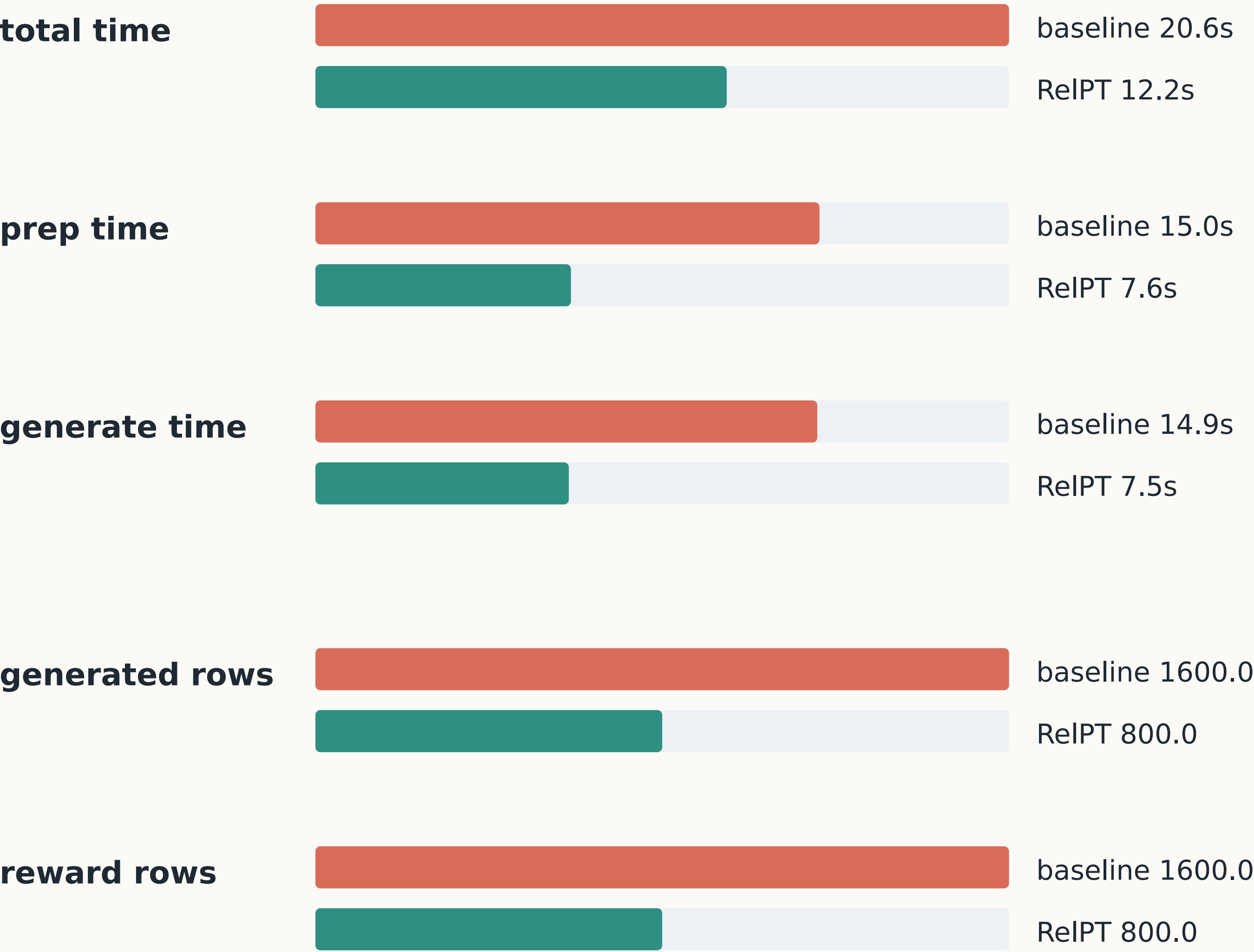
`trl\_gsm8k\_relpt\_ppo.py` keeps both arms on the same TRL PPOTrainer path. The RelPT arm caches prepared query, response, and reward rows for retry reuse; the baseline prepares them again. This isolates preparation efficiency from trainer changes.

Boundary

RelPT changes artifact planning and reuse. It does not change PPO loss.

200-step TRL PPO comparison

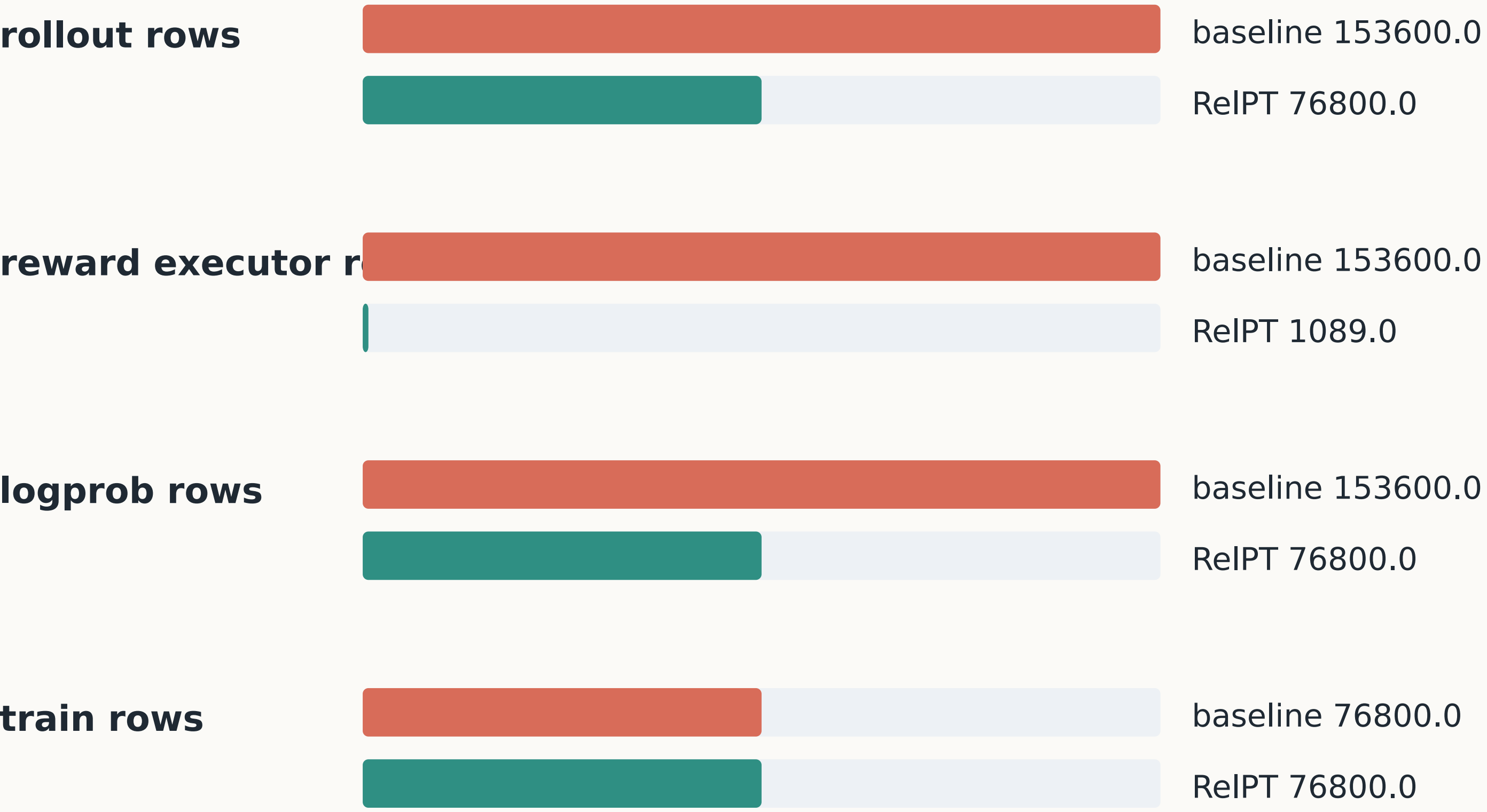
same trainer path, less repeated preparation



Observed savings

total 40.7% | prep 49.3% | generated rows 50.0%
model sshleifer/tiny-gpt2 | dataset pyarrow_arrow_cache | steps 200

Full RelPT prototype: large reward-cache reuse



Mechanism

At 200 steps, RelPT saves 50.0% of rollout rows, 99.3% of reward executor rows, and 50.0% of logprob rows. Train rows stay unchanged because PPO itself is intentionally the same black-box update.

How to read the evidence

Claim supported

RelPT reduces duplicated PPO preparation work by materializing and reusing artifacts.

Claim not made

The tiny model did not learn GSM8K math in this run; quality deltas are zero in the TRL job.

Why it matters

Real post-training pipelines pay heavily for rollout, reward, and logprob executors.

Prime_TTT link

The same relational planning idea fits the broader Prime_TTT view: optimize artifacts before the gradient sink.

Shortest conclusion: RelPT is a relational reuse layer for post-train